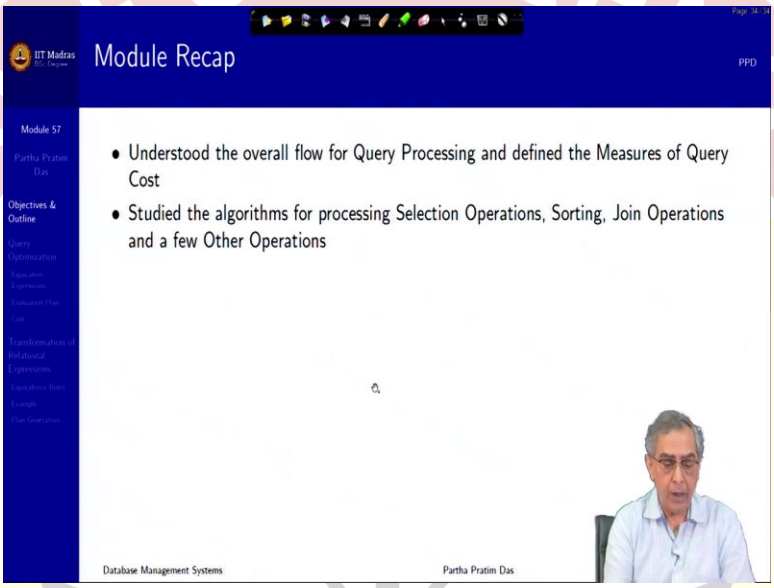
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology Kharagpur
Query Processing and Optimization/2: Optimization

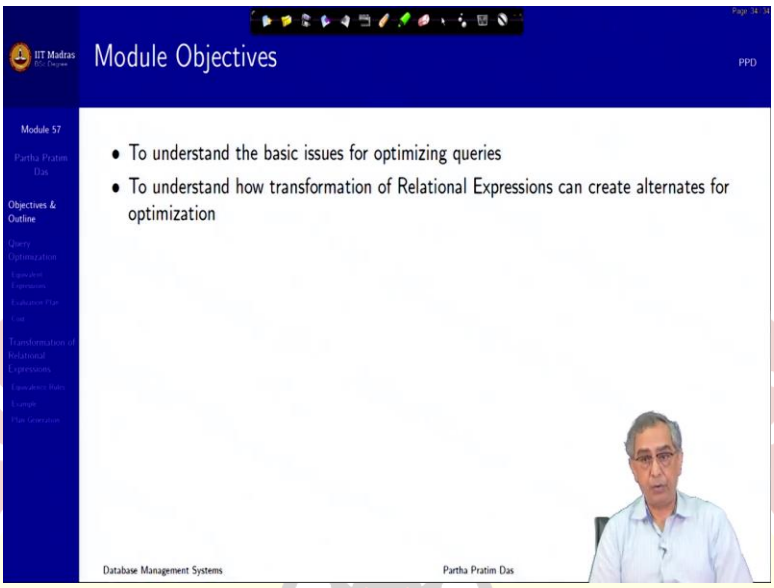
Welcome to Module 57 of Database Management Systems course in the IIT Madras, online BSc programme.

(Refer Slide Time: 00:26)



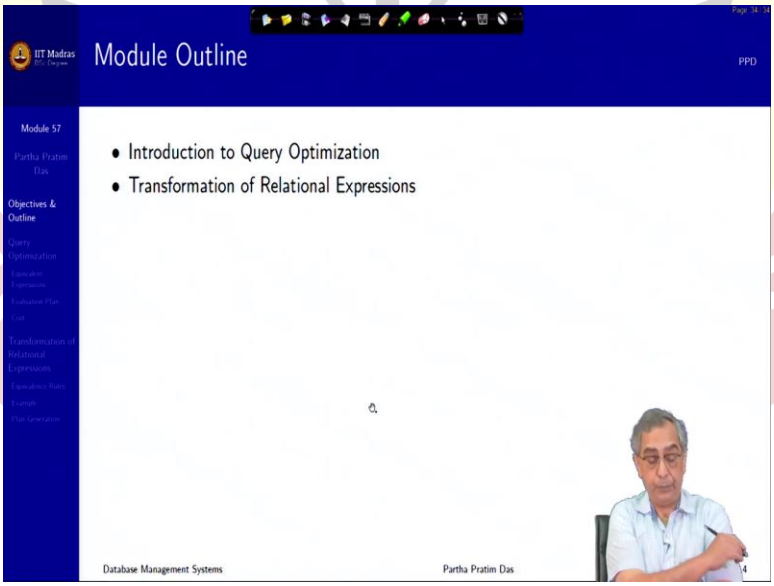
In the last module we started discussing about query processing and defined the measures of query cost, primarily in terms of disk transfers and disk seek and then we studied a few algorithms for processing have a variety of selection operations sorting, join operations and few other operations like duplicate removal, projection and aggregation and so, on.

(Refer Slide Time: 00:55)



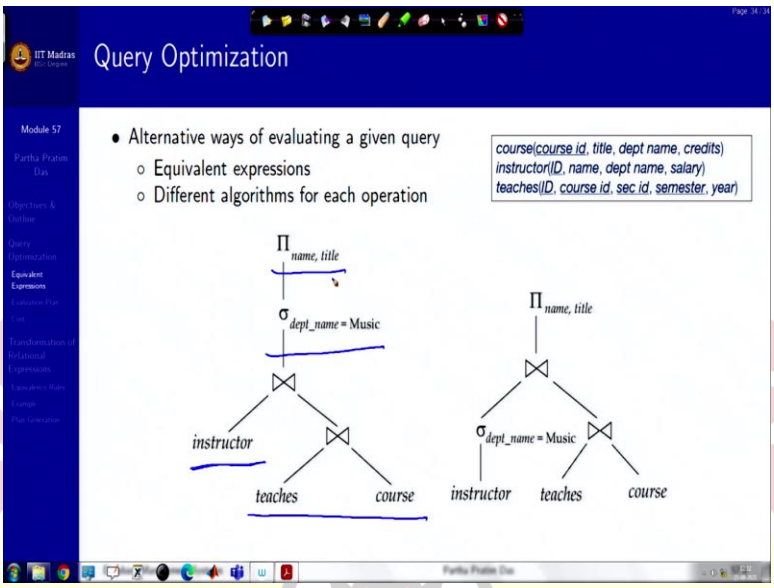
In the module, in this module, we will try to understand the basic issues of optimising the queries and how a relational expression can be transferred, transformed to create alternatives for optimization and a correct, not all of them will have to be correct and a best cost wise best query evaluation plan can be chosen.

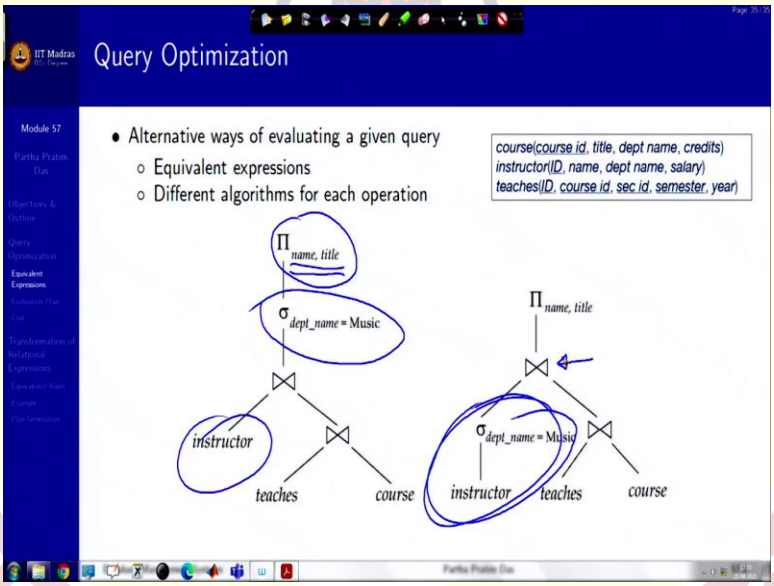
(Refer Slide Time: 01:24)



So, we talk about these two.

(Refer Slide Time: 01:31)





So, just to look at the query optimization, this is the, these are the two basic steps that you generate, once the translation is happened, you have a relational expression, you generate equivalent expressions and use different algorithms for each operation in those equivalent expressions and then you evaluate them for cost and choose the best one.

So, these expressions are now shown in terms of a tree structure. So, what you can easily see that what it is doing is it is making a first a join of teacher and course, teachers and course then I join with the instructor, then doing a selection based on department name is same as music and then finally, making a projection of name and title, so you can very easily read what will this query

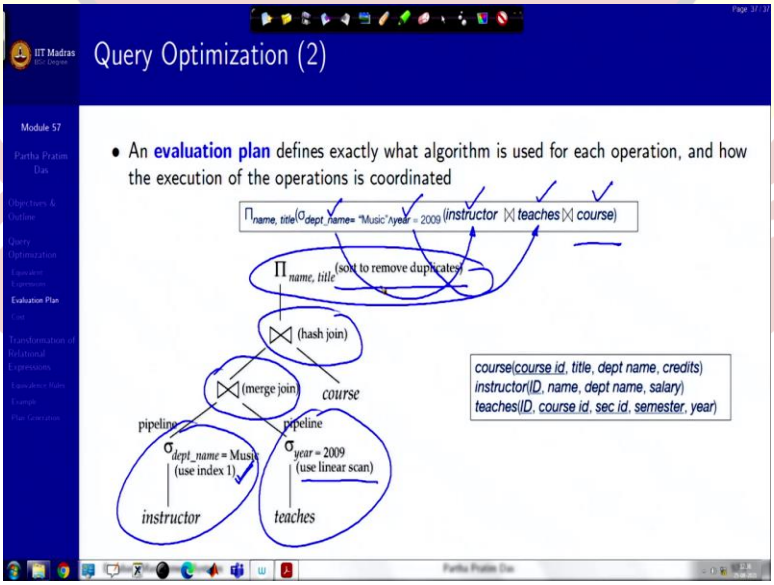
gave is, it will give us the name of the instructor and the course that he or she is teaching in for music department, nothing else, it is all.

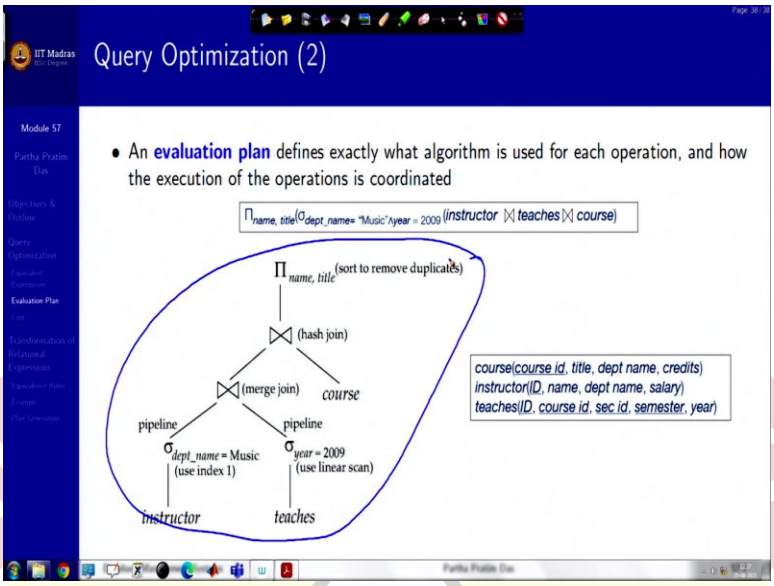
Now, this is this is in simple terms, how you will probably write the query or probably how there is a first translation will happen, but then we can see that well in their final result, the name and title of the instructor, I mean the name of the instructor that will come necessarily that instructor has to belong to the music department.

So, there is no point treating all other instructors who are not from music department in this whole process. So, instead of doing this filter here, we can do it right below here. What is advantage, advantage is a number of total instructors will be many and they are teaching a lot of courses, all these will, but once you have just consider on the music department, there will be few instructors.

So, this relation becomes much smaller compared to this relation and therefore, the subsequent join that you do here would be much smaller as well and you will benefit in terms of the, in terms of the operations that you have to do and finally, you do the projection. So, this is just an illustrative example to make you understand that how a simple equivalent transformation can give you a better way to evaluate a query.

(Refer Slide Time: 04:01)





So, this is how once this is fully worked out, then it is made into an annotated query tree, which is called the evaluation plan. So, we are talking about a little variant of the earlier query, where you want to find the names of the instructors and the courses they have taught in the year 2019, where the instructor belongs to the music department.

So, you have two conditions here and naturally three, relations are involved instructor as instructor information, courses have course information and teachers relates instructor with the course who is teaching what. So, you need to join all these three, this part you understand.

Now, how can this get optimised, as we had noticed earlier that department is specific for instructor. So, instead of doing the join and then taking out tuples, which has department as music, we can only take those instructors who belong to the music department. So, we do it here, we first take out the instructors with the music department and for the evaluation plan, we are also specifying what is the algorithm.

What is the algorithm? The algorithm is use index 1, why use index 1 I mean some index on this because instructor is more or less a static data. So, it is easy to build the index, it is not going to change very often and it will give you a very fast selection possible as we have seen in the terms of the selection algorithms we saw in the last module.

For the other condition e r, e is specific to teaching, the courses are again persistent, but the teaching for a particular year is dependent in that teacher’s relation only. So, we can also make the teacher's relation smaller, but in the absence of a, of an index we will use a linear scan for

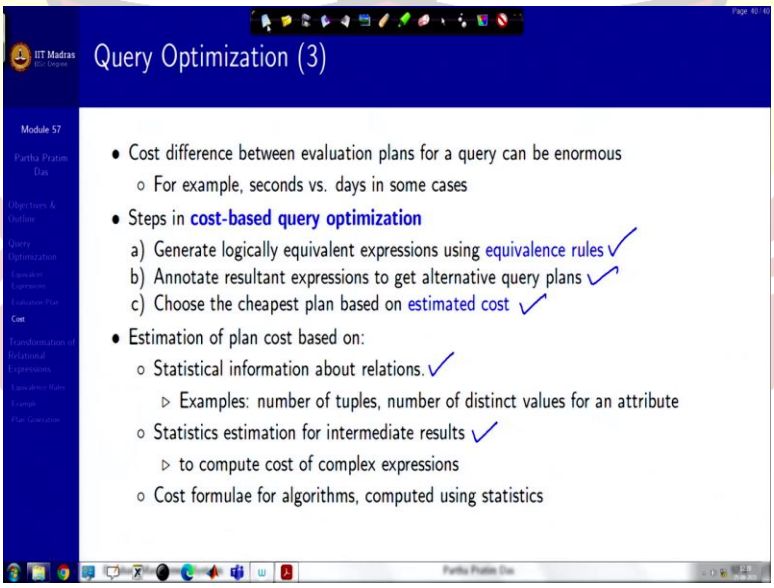
doing that. So, that is advice to the evaluation engine, that is how it is becoming an evaluation plan it is just not a of the presentation of the query expression or the relational expression.

So, then you join these two first, which is for which they use some merge join algorithm which is better than the kind of algorithms we have seen, you can read up specifically how merge join is done. But this is how it will and then it joins the second the last relation course using a hash join. I mean there is specific reasons of why you use hash join at this particular point because it is easy to hash the courses in a very compact form, because they have unique IDs to deal with.

And finally, you do the projection and hear what is being highlighted that you have to sort to remove duplicates. So, this entire thing, this entire tree with these annotations of algorithms is your evaluation plan and that is what the optimizer has to generate and pass on to the evaluation engine and naturally this will be generated based on different alternatives that you can have.

So, here I just used certain heuristic information like department occurs only in instructor. So, I can I can do that selection first or year is specific to teachers. So, I can do that filter first and so on. But in general, the system will have to have a strategy of exploring alternates and choosing the best one based on the cost estimates which are statistical in nature.

(Refer Slide Time: 08:22)



So, the cost difference between evaluation plans for a query can be enormous, I mean from seconds versus days and so on. So, the cost-based query optimization where we will have to first

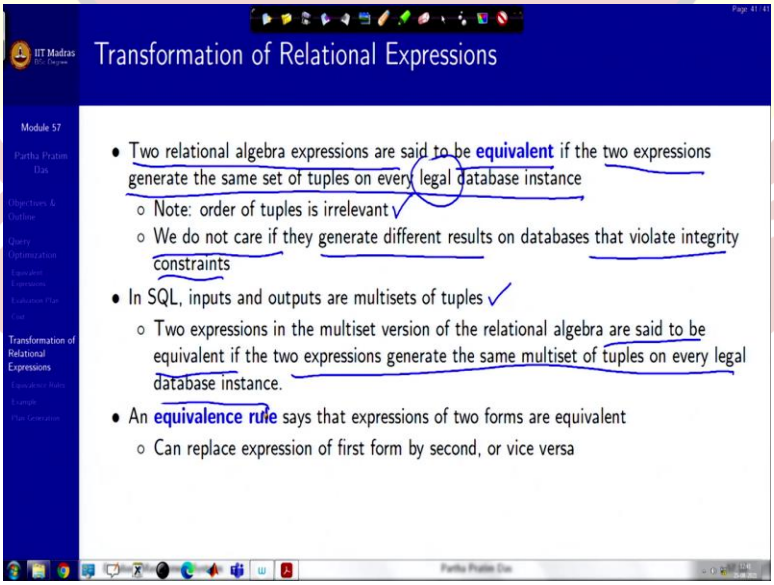
generate the alternates. So, how do you generate the alternates? So, we will define some equivalence rules for different operations of relational algebra.

So, that using those equivalence rules, you can convert one expression into another equivalent expression, then you will have to annotate the resultant expression to get alternative query plans. Because, I mean, once you have alternate expressions, you will know what is the order in which these expressions will have to be evaluated, what are the possible available index and so on. So, what is the algorithm that you can choose and so on.

And then choose this cheapest of these plans based on estimated cost, cost estimate could be statistical information about relation, it could be based on estimation on intermediate results, cost formula for algorithms computed using statistics and so on. So, all these different kinds of statistical information will be used to tune your optimise.

So, you can see that the same database design, the same query in case of two very different kinds of data. For example, even if the academic data, if you do this, the same query if you run for one university and if you run for a different university you may get a completely different optimization in terms of the evaluation plan. So, let us see what are these transformational equivalence rules.

(Refer Slide Time: 10:13)



Because we will have to do that. So, two relational algebra expressions are called equivalent, if the two expressions generate the same set of tuples on all legal database instances, only then they

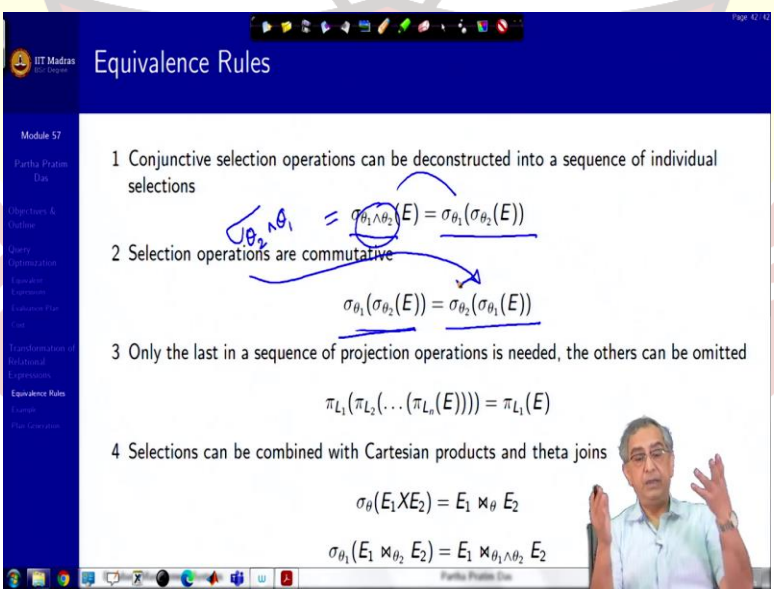
will be equivalent, that if they always generate the same output for every legal database instance, this term is important.

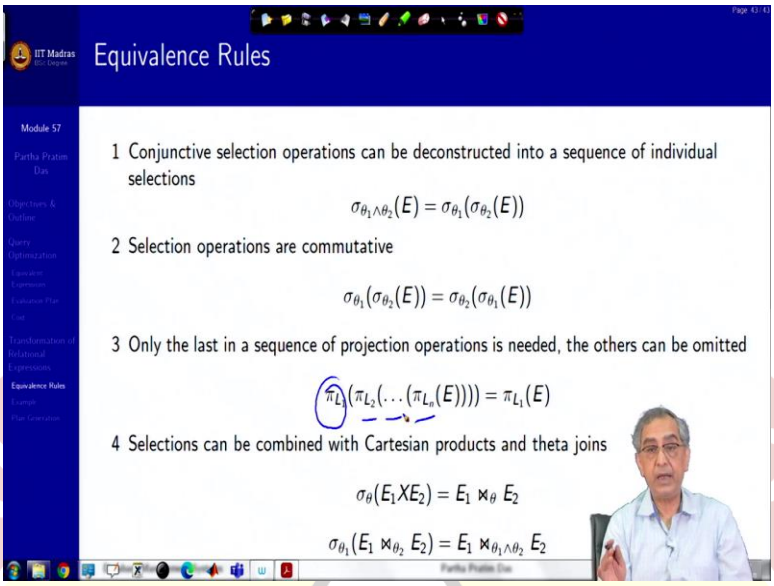
Now, here do keep in mind that the order of tuples in a relation, in general is not important. So, order of tuples could be different, it is a set if the same set then we accept them. Also, we would not care, we do not care if different results are generated for databases that violate the integrity constraint.

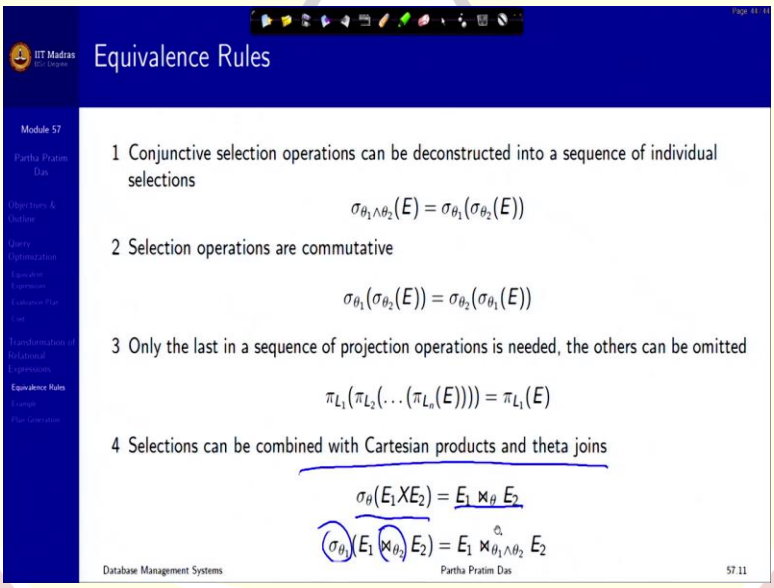
If the database violates integrity constraint, then it is possible that two expressions are equivalent, but generate different results. So, in SQL everything is multi set of tuples. So, two expression in the multiset version is said to be equivalent, if two expressions generate the same multi set of tuple on every legal database instance, so which is a little more tricky, because in relational algebra, we saw it is a set that we generate.

In SQL, there could be identical tuples existing in the same relation because it is a multiset. So, such identical tuples occurrences of such identical tuples will also have to be same between the two equivalent expression for every legal instance of the data. So, equivalence rules are formed, which say, how two expressions are equivalent by treating each and every operation one by one.

(Refer Slide Time: 12:06)





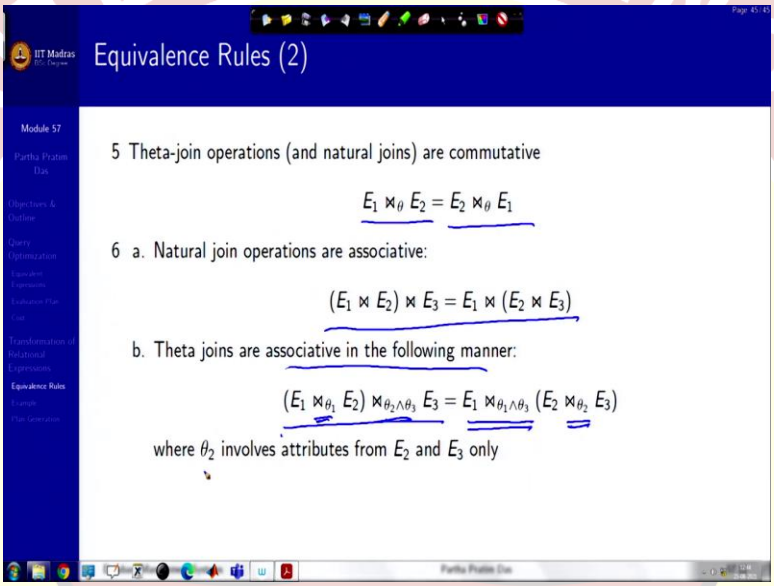


So, these are these are very intuitively easy rules, but still, if you have doubts, you should work it out, think over for example, if I have conjunction of two conditions, then it is applying the two conditions one after the other, which may be easier to evaluate. The selection is commutative. That is, if I have two conditions, then I can change their order, which is very easy to prove, because this ending is commutative.

So, this is equivalent to theta 2 and theta 1 and if I do that, then by this rule, I can say that this is same as this. So, all of these are equivalent, the only last in a sequence of projection matters, if you are projecting then that column will have to exist in the relation. So, if you are making multiple projections, you are basically reducing the number of columns.

So, is the final one that matters, everything else can be ignored, these are the equivalent rules, the selection combined with Cartesian product and theta join. So, if you do a Cartesian product and then do a selection by theta is the same as a theta join and basically is the definition and if you have two conditions, that is, if you are doing a theta join and then do a selection, you can combine these two conditions into the same theta join. So, these are I mean, these are very intuitively clear equivalences, but we are just noting them down.

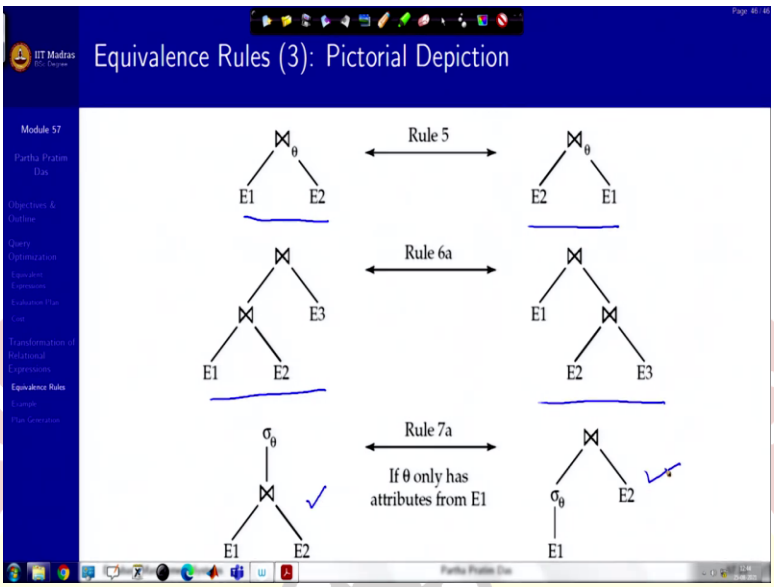
(Refer Slide Time: 13:45)



For a theta join and natural join, they are commutative that is I can change the order of the relations, what which one will be outer which one will be inner that can be easily chased, natural join is associative, that if there are three to be joined, I can do the first two and join the third with the result or I can join the second and third and join first with that result.

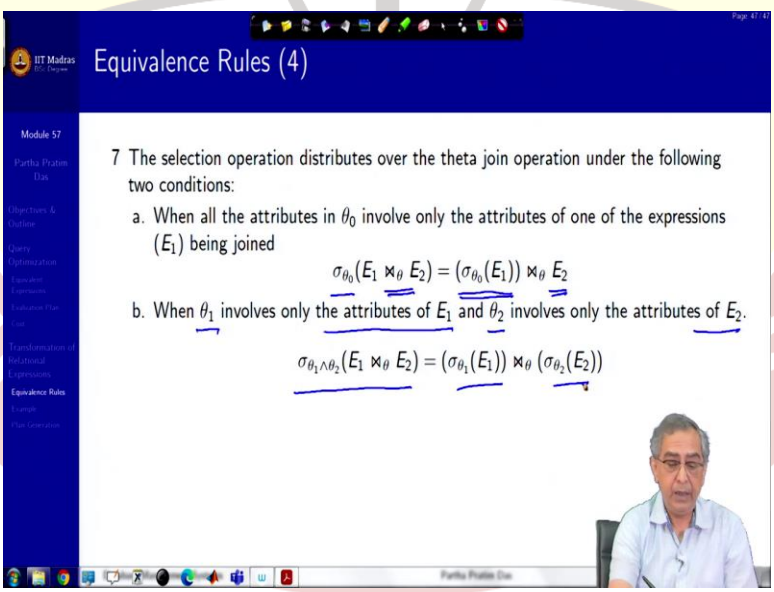
Theta join has some nuances in the associativity. For example, if you do a theta join here and another theta join here with these conditions, then you can have that theta 1 and theta 3 ended and you do theta 2 join with this. Think over it. But you will see that this is possible where theta 2 involves attributes of E2 and E3 only. If it does not then you cannot do this kind of equivalence.

(Refer Slide Time: 14:51)



So, often it becomes more comprehensible if you draw them in terms of the expression tree like the way we said the commutativity of theta join, this is associativity of natural join and this is the last rule that we saw, you can draw them in terms of these thee for easier understanding.

(Refer Slide Time: 15:20)

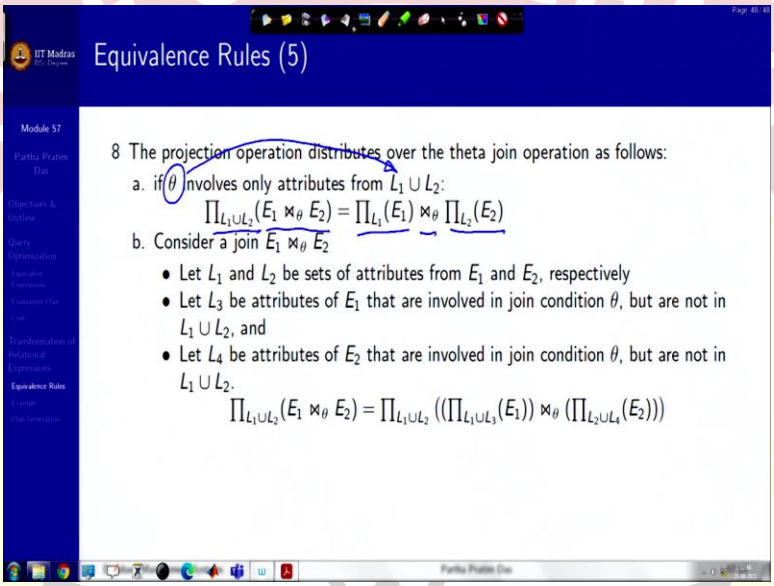


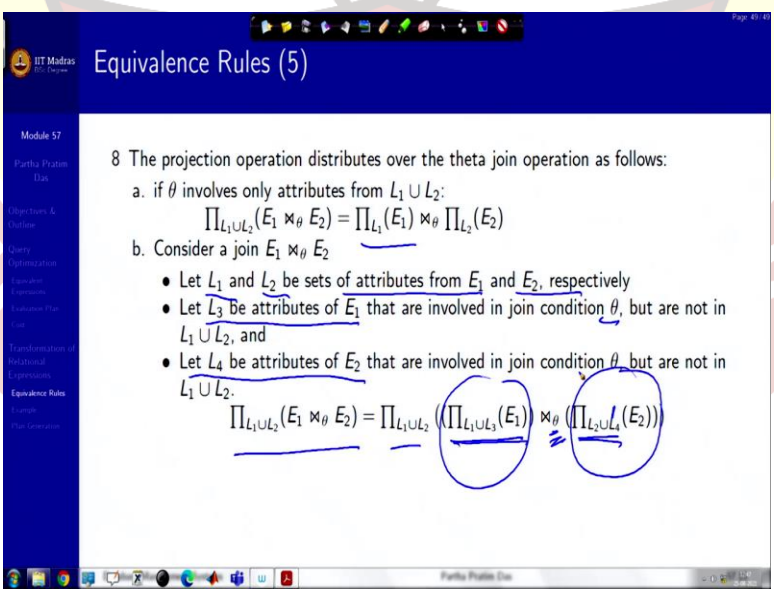
Some more, the selection operation will distribute over theta join. So, selection operation distributes over theta join, in a way that you can do the selection on the first relation and then do the theta join with that second, it is quite obvious. When theta 1 involves only attributes of E1,

and theta 2 involves only attributes of E2, then in this condition, you can distribute the selection on both of them individually.

These are all I mean, you can mathematically prove we can write down the calculus predicate calculus formula corresponding to this and show that they are equivalent, but they are intuitively absolutely clear. But keep in mind, the different conditions that hold for these equivalences.

(Refer Slide Time: 16:19)





Projection operation distributes over theta join, how you are projecting, you have made a theta join and then you are projecting on say attributes L1 union L2 where the theta the condition involves only these attributes, it does not involve other attributes, it could but if it does not, then

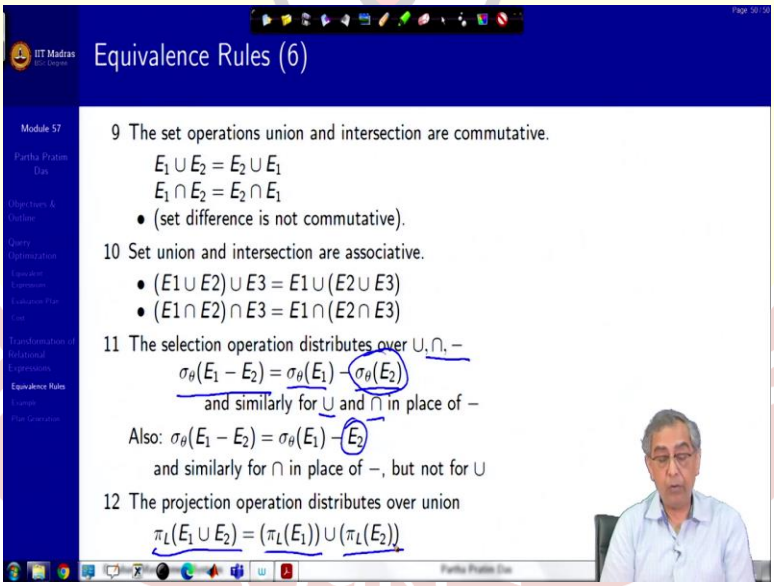
it is enough to project the first one on L1, second one on L2 and then do that. Naturally, this will get smaller, it is easy to see why they are equivalent.

So, L1, L 2 the set of attributes from E1 and E2 respectively. Now, let us say L3 be attributes of E1 that are involved in theta, but not in L1 union L2 some more attributes of E1 are involved in the theta and L4 are some more attributes of E2 that are also involved in theta, but not in this union set.

Now, what will this look like? Now, since for if L3 comes from one your projection cannot only be on l one, it will have to be an L1 union L3 because otherwise you will not be able to apply that to join later on, because you will miss those attributes. Similarly, this will have to be on L2 union L4 because otherwise, the attributes L4 will not be available for evaluating the theta.

So, once you have done that, you have L1 union L3 union L2 union L4 all of these, so you have to then do a projection, but obviously these projections might significantly reduce the size of the relation because you are stripping off all other attributes.

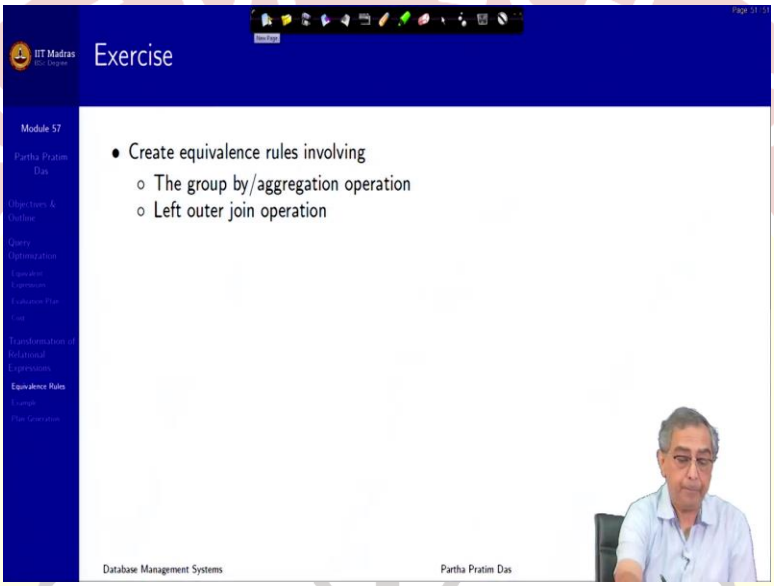
(Refer Slide Time: 18:20)



Set operations are commutative union and intersection not the difference as you know, A minus B is not same as B minus A. Set union and intersections are associative again that is well known, the selection operation distributes over, so if you do a selection of E1 difference E2, you can do that by selecting this and by selecting this and same thing will apply for union and intersection also.

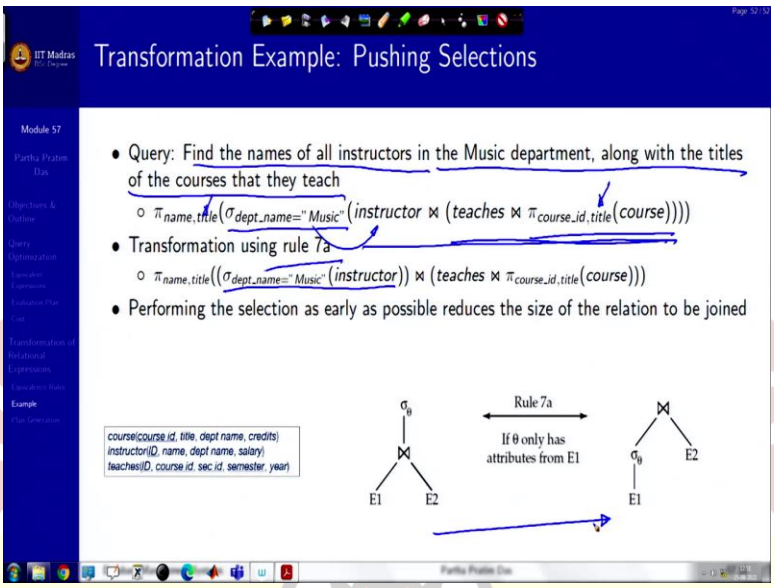
Actually particularly, what you can do is, you may not do the selection either, you can just use the same relation because you are doing a difference, you are going to take away. So, whether you do the selection, second selection or not is a matter of choice, all of these are equivalent. If you do projection that distributes over union, if you do a projection on the union, it is okay to do projection and then take the union it is quite simple to understand.

(Refer Slide Time: 19:26)



So, I will leave a few as exercise for drawing the equivalence rules. For example, create the equivalence rules for group by or for aggregation operations, so that you can generate the alternates or for say, left outer join operation and so on.

(Refer Slide Time: 19:47)

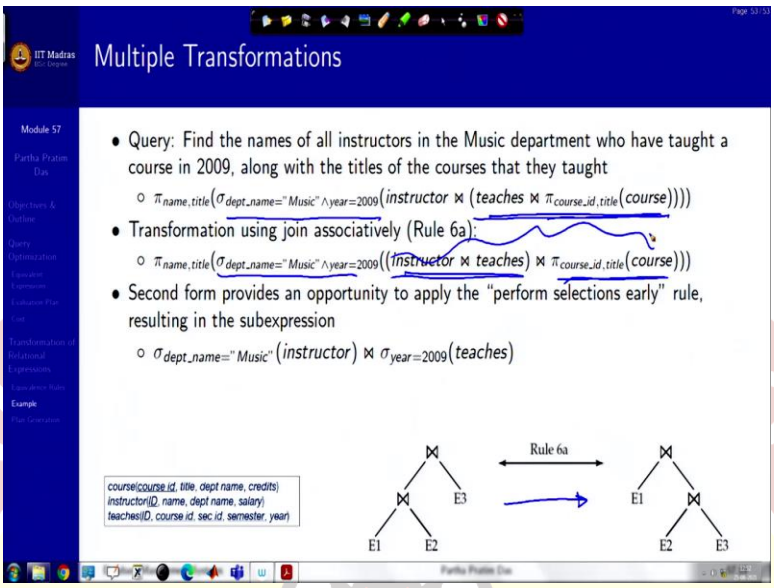


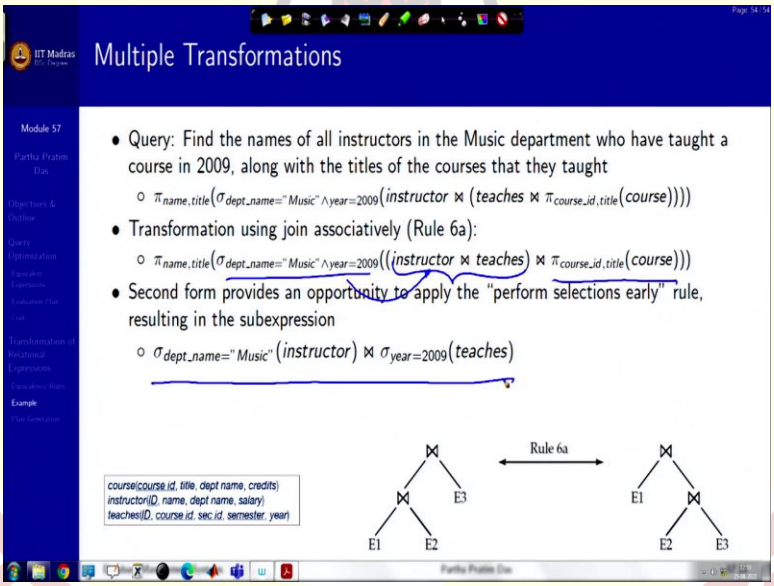
Now, so what we have now, we have a lot of different alternates that can be generated. So, using these equivalence rules. So, let us see the effect on some of our examples that we are seeing, so find all instructors names in the music department along with the title of the course that they teach, what we considered before.

Now, this is we say is in the course, we need course ID to join it with teachers, that is a common we need title at the end in the output. So, rest of the course details can be dropped, then you join with teachers, join with instructor, select based on department name music and finally do the projection.

But you can use rule 7 A, in which while doing a, while doing a selection on natural join, you can restrict the selection to the first relation only. So, by that you move that in here and this rule is what is being applied and with this, it will get even more smaller performing the selection as early as possible, reduces the size of the relation to be joined, very basic thumb rule. So, this is pushing relation is a very good idea.

(Refer Slide Time: 21:33)





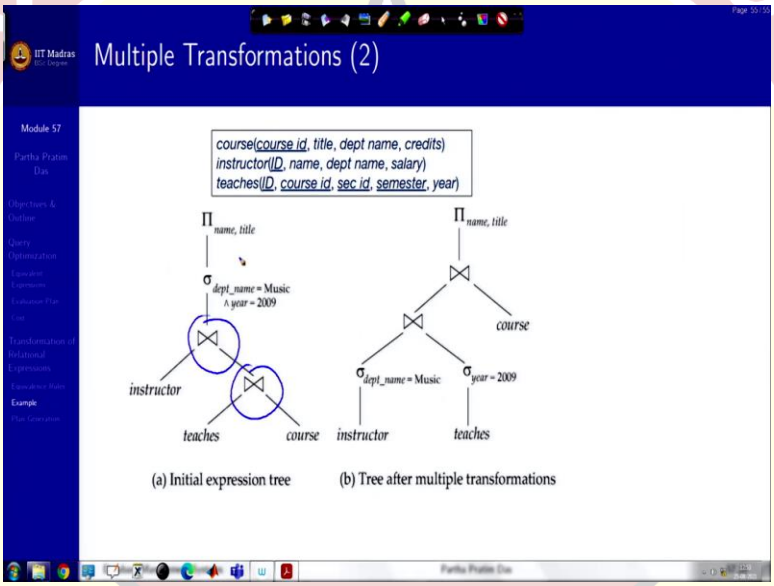
You may need multiple transformations, the other query, find the names of all instructors in the music department who have taught a course in 2009, along with the title of the course that they taught, we saw this. So, this remains same, here you have two conditions and it conjunction of conditions.

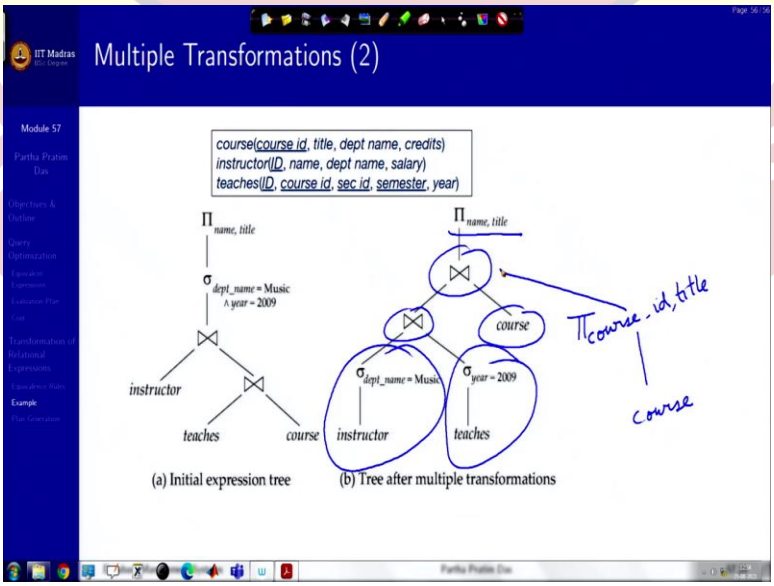
So, as we have that, we can use rule 6 A, to change the associativity of the joint. So, if we change that, then instead of doing this join first we are doing this join first. Now, if we are doing this join, we do not, we then do the second join. Now, again, we can perform the selection first rule, as we have seen, we can push the selection.

So, instead of doing the selection on this entire join, we can simply push this on the first relation which is the join of instructor and teachers and then join with the this, as we push that, then we have those rules which tell us that I can just do selection based on music on that condition on instructor, because it does not involve any attribute of teachers.

Similarly, I can do the selection for e r is equal to 2009, only on the teachers because it does not involve any attribute from the instructor. So, that results in both of these relations first getting performing a selection, making the site really small and then doing the subsequent join.

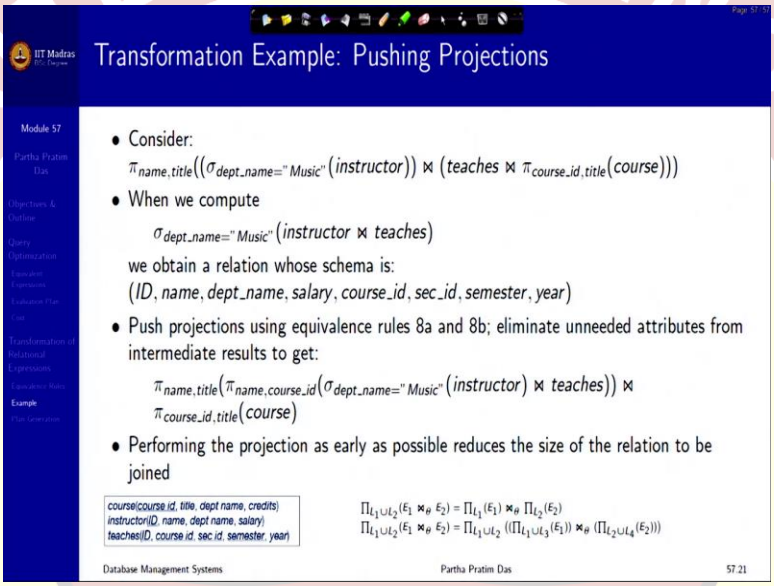
(Refer Slide Time: 23:51)





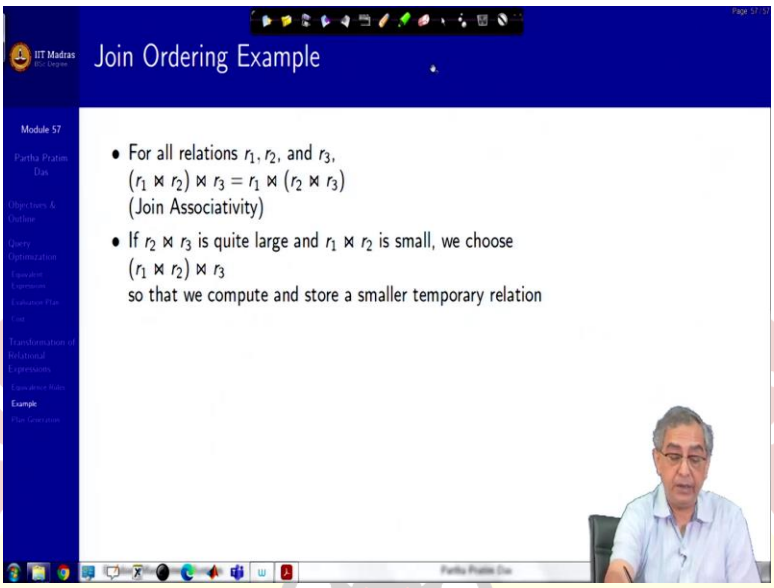
So, this was a original tree, it was doing a first join here, second join here, then the selection then the projection. Now what you are doing, you push this down, you are doing a selection on the instructor, you are doing a selection on the year, then doing this join you have changed by the associativity rule and then joining by the course and doing the name title. Actually, what you can, you can you can further do is you can have course ID tittle, it is gonna be a projection the course also. You do not lose any data it will become even smaller.

(Refer Slide Time: 24:54)



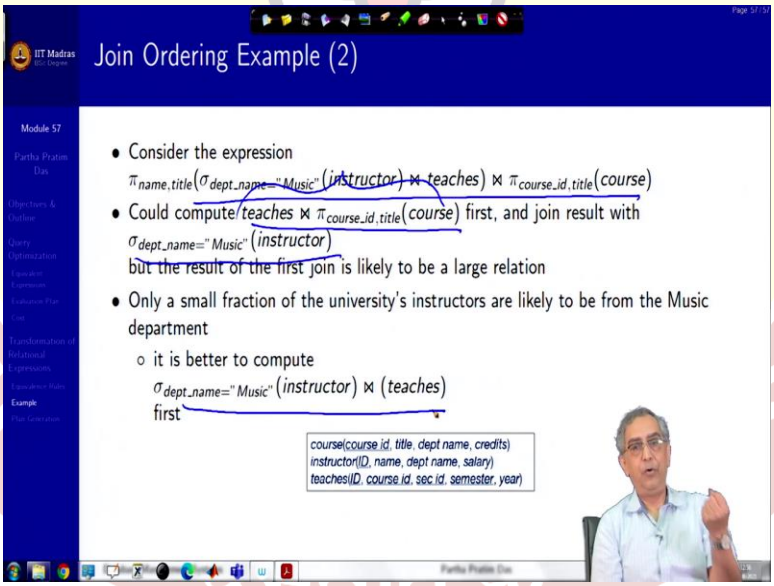
So, some more examples I think you have, we have already discussed this, but it is performing projection as early as possible, we are trying to illustrate that rule just go through it and you should understand it easily.

(Refer Slide Time: 25:14)



In terms of join you use the associativity based on which part is expected to be smaller. So, you do that part because you can store that more easily.

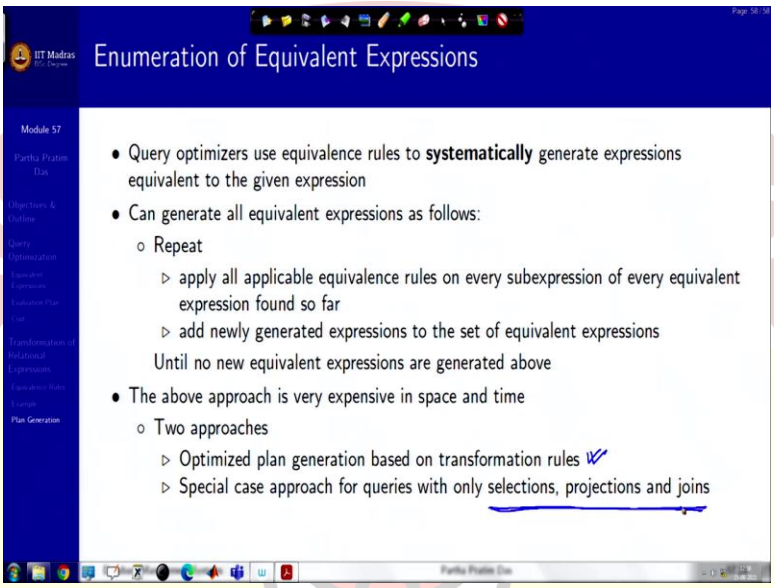
(Refer Slide Time: 25:31)



So, when we consider the expression, we could compute teaches with the course ID first and then join the result here. But this teaches times, teaches natural join you know projection of course, is expected to generate larger result how do I know, statistical information. We can intuitively say we can heuristically say as human beings we can think, because courses of several years in the, in the teaches that exist and there are naturally the course list is also very large.

And so, it will be it will be quite some, but if I can take out only courses which is taught by instructors of music department, which is what this does, then that will not be very large and then taking a join with the courses would just, for just taking out the title would be appropriate.

(Refer Slide Time: 26:48)

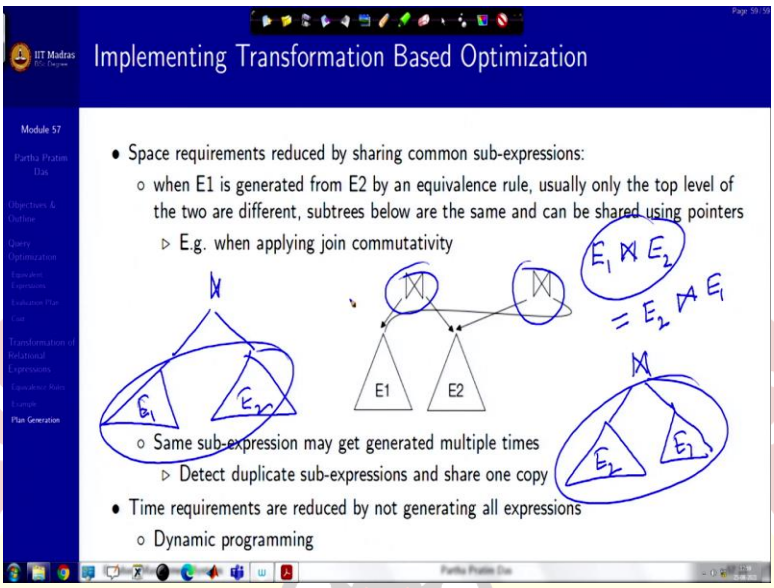


So, these are the different ways to generate alternate. So, the actual query evaluation plan will get generated by the query optimizer using these equivalence rules in a systematic manner, because you have to, there is no known as to how many equivalent expressions should be there. We can generate all of them by enumeration, apply all applicable equivalence rules on all sub expressions of all equivalent expressions found so far and keep on generating that till, till you do not have anything new getting added to the set.

Brute force method, which will obviously be kind of exponential and therefore, very expensive both in terms of time and in terms of space, obviously you are optimising the query. So, the time taken to optimise cannot be so large, that the benefit of optimization is lost or the query actually slows down.

Because you might be able to find a very optimal way to do the query, but the time you have taken to optimise is so large that your net effect is null. So, often you use the, you try to generate the optimised plan on the transformation rule and take different kinds of special case approach for the very common and widely used operations like selection projection and join.

(Refer Slide Time: 28:32)

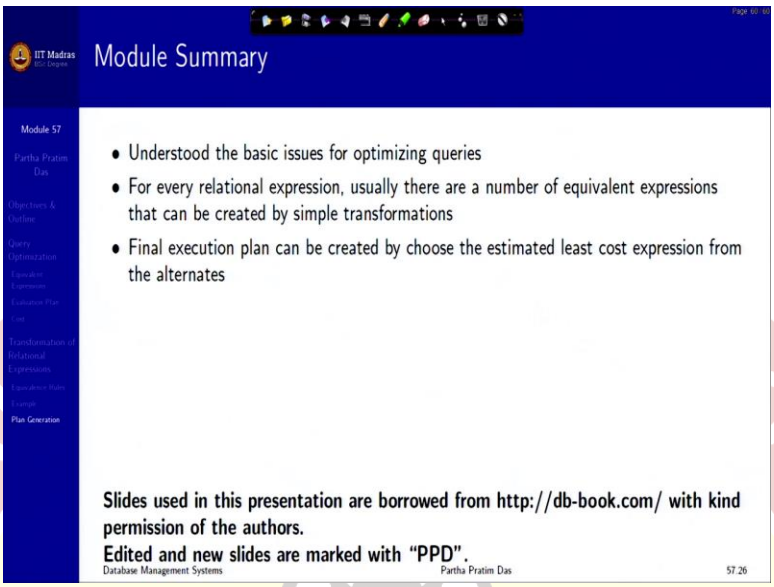


So, if you think about that, then there are different ways you can handle things better, for example space, you can optimise by not duplicating the same expression, for example you, we have the rule that E1 is same as commutativity. So, which means that I have a join and I have E1 here, I have E2 here and they could be potentially large expressions. This is for the left-hand side and I will similarly have further right-hand side. E2 here and E1 here.

But instead of making two copies of this, for this alternate, I could just manage by being kind of a multi tree pointers. So, the same sub expression may get generated multiple times, I can have a smart algorithm to detect duplicate sub expression and share only one copy. So, there are, we do not have the opportunity to get into depth of these discussions, but I just wanted to give you a glimpse of what are the issues that are involved and above all time requirements are often drastically reduced by not generating all expressions, but using some kind of a dynamic programming. So, you keep on.

So, you as you keep on generating alternate expressions you keep on tracking, what is their current cost, so you always have the best current cost option. So, you dynamically keep on checking as to how much, where all you need to improve on the equivalent expression on which all will never give you a better expression and keep on pruning that, I mean typically the way the dynamic programming Tableau method works.

(Refer Slide Time: 30:59)



So, that is to summarise this is just a glimpse of optimising, optimization techniques for queries and generating a good evaluation plan for a given query. Thank you very much for your attention. We will meet in the next module.